УНИВЕРСИТЕТ ИТМО

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 Программная инженерия

Дисциплина «Проектирование вычислительных систем»

Лабораторная работа №4

Вариант 1

Студенты

Нодири Хисравхон

Руотси Ян

P3431, P3433

Преподаватель

Пинкевич В. Ю.

Санкт-Петербург, 2025г.

Задание лабораторной работы

Разработать программу, которая использует интерфейс I2C для считывания нажатий
кнопок клавиатуры стенда SDK-1.1.

Подсистема опроса клавиатуры должна удовлетворять следующим требованиям:

- реализуется защита от дребезга
- нажатие кнопки фиксируется сразу после того, как было обнаружено, что кнопка нажата (с учетом защиты от дребезга), а не в момент отпускания кнопки; если необходимо, долгое нажатие может фиксироваться отдельно
- кнопка, которая удерживается дольше, чем один цикл опроса, не считается повторно нажатой до тех пор, пока не будет отпущена (нет переповторов)
- распознается и корректно обрабатывается множественное нажатие (при нажатии более чем одной кнопки считается, что ни одна кнопка не нажата, если это не противоречит требованиям к программе)
- всем кнопкам назначаются коды от 1 до 12 (порядок на усмотрение исполнителей).

Программа должна иметь два режима работы, переключение между которыми
производится по нажатию кнопки на боковой панели стенда:

- режим тестирования клавиатуры
- прикладной режим.

Уведомление о смене режима выводится в UART.
В режиме тестирования клавиатуры программа выводит в UART коды нажатых кнопок.
В прикладном режиме программа обрабатывает нажатия кнопок и выполняет действия в соответствии с вариантом задания.

Вариант задания

Реализовать «музыкальную клавиатуру» с помощью излучателя звука.
Существует девять стандартных октав от субконтроктавы (первая по порядку) до пятой октавы (девятая по порядку) (более подробно об октавах см. в специализированных источниках). Частоты нот в соседних октавах отличаются ровно в два раза и растут с номером октавы.
Частоты для первой октавы (пятая по порядку):

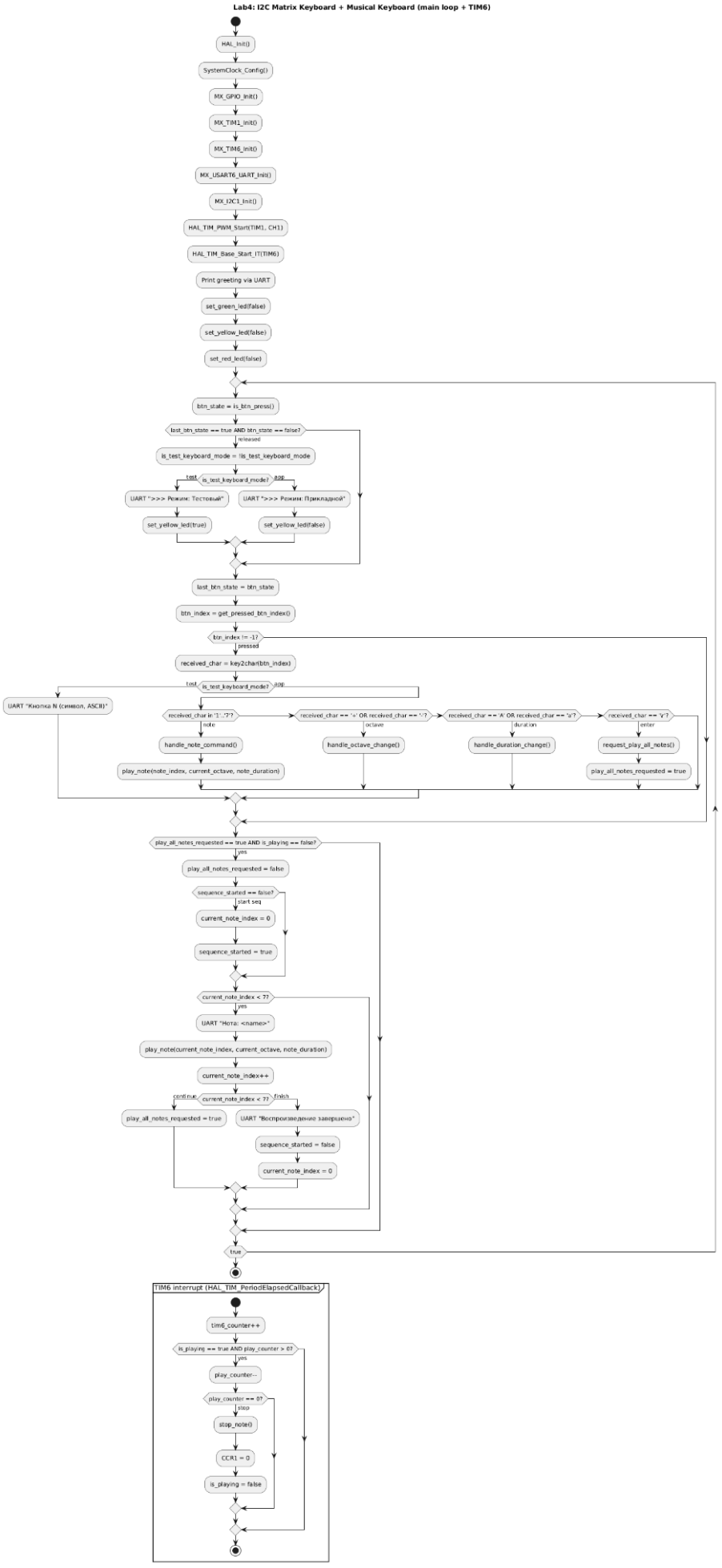
Нота	Частота, Гц
До	261,63
Ре	293,67
Ми	329,63
Фа	349,23
Соль	392,00
Ля	440,00
Си	493,88

Символ	Действие
«1» – «7»	Воспроизведение одной ноты (от «до» до «си») текущей октавы с текущей длительностью звучания. Начальные значения: первая октава (пятая по порядку), длительность 1 с.
«+»	Увеличение номера текущей октавы (максимальная – пятая).
«-»	Уменьшение номера текущей октавы (минимальная – субконтроктава).
«A»	Увеличение длительности воспроизведения ноты на 0,1 с (максимум – 5 с).
«a»	Уменьшение длительности воспроизведения ноты на 0,1 с (минимум – 0,1 с).
«Enter»	Последовательное воспроизведение всех нот текущей октавы с текущей длительностью без пауз.

По вводу каждого символа в UART должно выводиться сообщение:

- для символов «1» – «7», «Enter»: какая нота какой октавы и с какой длительностью проигрывается
- для символов настройки: новые значения номера октавы и длительности звучания ноты;
- для символов, не перечисленных в таблице выше: сообщение «неверный символ» и его код.

Блок схема кода


Lab4: I2C Matrix Keyboard + Musical Keyboard (main loop + TIM6)
HAL_Init()
SystemClock_Config()
MX_GPIO_Init()
MX_TIM1_Init()
MX_TIM6_Init()
MX_USART6_UART_Init()
MX_I2C1_Init()
HAL_TIM_PWM_Start(TIM1, CH1)
HAL_TIM_Base_Start_IT(TIM6)
Print greeting via UART
set_green_led(false)
set_yellow_led(false)
set_red_led(false)
btn_state = is_btn_press()
last_btn_state == true AND btn_state == false?
released
is_test_keyboard_mode = !is_test_keyboard_mode
is_test_keyboard_mode?
test
app
UART ">>> Режим: Тестовый"
UART ">>> Режим: Прикладной"
set_yellow_led(true)
set_yellow_led(false)
last_btn_state = btn_state
btn_index = get_pressed_btn_index()
btn_index != -1?
pressed
received_char = key2char(btn_index)
is_test_keyboard_mode?
test
app
UART "Кнопка N (символ, ASCII)"
received_char in '1'..'7'?
note
handle_note_command()
play_note(note_index, current_octave, note_duration)
received_char == '+' OR received_char == '-'?
octave
handle_octave_change()
received_char == 'A' OR received_char == 'a'?
duration
handle_duration_change()
received_char == 'X'?
enter
request_play_all_notes()
play_all_notes_requested = true
play_all_notes_requested == true AND is_playing == false?
yes
play_all_notes_requested = false
sequence_started == false?
start seq
current_note_index = 0
sequence_started = true
current_note_index < 7?
yes
UART "Нота: <name>"
play_note(current_note_index, current_octave, note_duration)
current_note_index++
current_note_index < 7?
continue
finish
play_all_notes_requested = true
UART "Воспроизведение завершено"
sequence_started = false
current_note_index = 0
true
TIM6 interrupt (HAL_TIM_PeriodElapsedCallback)
tim6_counter++
is_playing == true AND play_counter > 0?
yes
play_counter--
play_counter == 0?
stop
stop_note()
CCR1 = 0
is_playing = false

Исходный код

Объявление переменных

```
#define KB_I2C_ADDRESS (0xE2)
#define KB_I2C_READ_ADDRESS ((KB_I2C_ADDRESS) | 1)
#define KB_I2C_WRITE_ADDRESS ((KB_I2C_ADDRESS) & ~1)
#define KB_INPUT_REG (0x0)
#define KB_OUTPUT_REG (0x1)
#define KB_CONFIG_REG (0x3)
#define KB_KEY_DEBOUNCE_TIME (200)

#define ROW1 0xFE
#define ROW2 0xFD
#define ROW3 0xFB
#define ROW4 0xF7

const uint16_t row_size = 3;
uint32_t keys[] = {
      '1', '2', '3',
      '4', '5', '6',
      '7', '+', '-',
      'A', 'a', '\r'
};
const uint32_t oct_size = 7;
uint64_t freqs[] = {
      16350, 18475, 20610, 21820, 24500, 27500, 30870,
      32700, 36950, 41210, 43650, 49000, 55000, 61740,
      65410, 73910, 82410, 87310, 98000, 110000, 123480,
      130820, 147830, 164810, 174620, 196000, 220000, 110000,
      261630, 293670, 329630, 349230, 392000, 440000, 493880,
      523260, 587340, 659260, 698460, 784000, 880000, 987760,
      1046520, 1174680, 1318500, 1396900, 1568000, 1720000, 1975500,
      2093000, 2349200, 2637000, 2739800, 3136000, 3440000, 3951000,
      4186000, 4698400, 5274000, 5479600, 6272000, 6880000, 7202000
};
uint32_t note_index = 0;
uint32_t octave = 4;
uint32_t duration = 1000;
uint8_t is_all_playing = 0;
char* note_name[] = {"До", "Ре", "Ми", "Фа", "Соль", "Ля", "Си"};

uint8_t is_writing_now = 0;
char read_buffer[100];
char write_buffer[100];
char* cur_process_char = read_buffer;
char* cur_read_char = read_buffer;
char* transmit_from_pointer = write_buffer;
char* write_to_pointer = write_buffer;

uint32_t key_press_delay = 200;
uint32_t side_press_delay = 250;
uint32_t last_press_time = 0;
uint32_t last_side = 0;

uint8_t is_test = 0;

int32_t last_index = -1;
```

Вспомогательные функции

```
char* concat(char *s1, char *s2) {
    char *result = malloc(strlen(s1) + strlen(s2) + 1);
```

```
    strcpy(result, s1);
    strcat(result, s2);
    return result;
}

void next(char **pointer, char *buffer) {
    if(*pointer >= buffer + 100){
        *pointer = buffer;
    }
    else {
        (*pointer)++;
    }
}

void write_char_to_buff(char c) {
   *write_to_pointer = c;
   next(&write_to_pointer, write_buffer);
}

void write(char* str) {
    char* str_with_newlines = concat("\r\n", str);
    //str_with_newlines = concat(str_with_newlines, "\r\n");
    int size = sizeof(char)*strlen(str_with_newlines);
    for(size_t i = 0; str_with_newlines[i] != '\0'; i++) {
      write_char_to_buff(str_with_newlines[i]);
   }
}

int is_number(char* str) {
    for (size_t i = 0; str[i] != '\0'; i++) {
      if (!isdigit(str[i])) return 0;
    }
    return 1;
}

void restart_timer() {
   TIM6->CNT = 0;
}

void mute() {
   TIM1->CCR1 = 0;
}

void unmute() {
   TIM1->CCR1 = TIM1->ARR / 2;
}

 void set_frequency(uint64_t freq_millis) {
   TIM1->PSC = ((2 * HAL_RCC_GetPCLK2Freq()) / (2 * (TIM1->ARR) * (freq_millis
/ 1000))) - 1;
}

int get_frequency_by_index(uint32_t index) {
   return freqs[index + (octave * oct_size)];
}

void play(uint32_t index) {
   int freq = get_frequency_by_index(index);
   set_frequency(freq);
   restart_timer();
   unmute();
}

void start_playing() {
   is_all_playing = 1;
   note_index = 0;
```

```
    char answer[100];
    sprintf(answer, "Note: %s, octave: %d", note_name[note_index], octave+1);
    write(answer);
    play(note_index);
}

void duration_decrease(){
    if (duration > 100) {
        duration -= 100;
        TIM6->ARR = duration*10;
        restart_timer();
    }
}

void duration_increase() {
    if (duration < 5000) {
        duration += 100;
        TIM6->ARR = duration*10;
        restart_timer();
    }
}

void switch_mode() {
    if (is_test) {
        is_test = 0;
    } else is_test = 1;
}

void check_button() {
    uint32_t now = HAL_GetTick();
    if ((now - last_side) > side_press_delay) {
        if (HAL_GPIO_ReadPin(GPIOC,GPIO_PIN_15) == 0) {
            last_side = now;
            switch_mode();
            if (is_test) {
                write("Режим тестирования включен");
            } else write("Режим тестирования выключен");
        }
    }
}

void process_symbol(char current) {
    char answer[100];
    if (current >= '1' && current <= '7') {
        note_index = current - '1';
        sprintf(answer, "Note: %s, octave: %d", note_name[note_index], octave+1);
        write(answer);
        play(note_index);
    } else {
        switch (current) {
        case '+':
            octave++;
            if (octave > 8) octave = 8;
            sprintf(answer, "New octave is: %d", octave+1);
            write(answer);
            break;
        case '-':
            if (octave != 0) {
                octave--;
            }
            sprintf(answer, "New octave is: %d", octave+1);
            write(answer);
            break;
        case 'a':
            duration_decrease();
            sprintf(answer, "New duration is: %d millis", duration);
```

```
            write(answer);
            break;
        case 'A':
            duration_increase();
            sprintf(answer, "New duration is: %d millis", duration);
            write(answer);
            break;
        case '\r':
            start_playing();
            break;
        default:
            sprintf(answer, "Incorrect symbol %u", current);
            write(answer);
            break;
        }
    }
}


HAL_StatusTypeDef init_kb( void ) {
    HAL_StatusTypeDef ret = HAL_OK;
    uint8_t buf;

    buf = 0;
    ret = HAL_I2C_Mem_Write(&hi2c1, KB_I2C_WRITE_ADDRESS, 0x02, 1, &buf, 1,
100);
    if( ret != HAL_OK ) {
        return ret;
    }

    buf = 0;
    ret = HAL_I2C_Mem_Write(&hi2c1, KB_I2C_WRITE_ADDRESS, 0x01, 1, &buf, 1,
100);

    return ret;
}

int get_index() {
    int index = -1;
    uint8_t reg_buffer = ~0;
    for (int row = 0; row < 4; row++) {
        uint8_t buf = ~((uint8_t) (1 << row));

        int16_t n_key = 0x00;
        HAL_StatusTypeDef ret = HAL_OK;

        ret = init_kb();

        ret = HAL_I2C_Mem_Write(&hi2c1, KB_I2C_WRITE_ADDRESS, 0x03, 1, &buf, 1,
100);
        HAL_Delay(10);
        ret = HAL_I2C_Mem_Read(&hi2c1, KB_I2C_READ_ADDRESS, 0x00, 1, &buf, 1,
100);

        uint8_t mask = 0x7;
        n_key = (~(buf>>4)) & mask;

        switch (n_key) {
        case 0x1:
            if (index != -1) return -1;
            index = row*row_size;
            break;
        case 0x2:
            if (index != -1) return -1;
            index = (row*row_size) + 1;
            break;
```

```
      case 0x4:
         if (index != -1) return -1;
         index = (row*row_size) + 2;
         break;
      }
   }
   return index;
}

int32_t get_index_checked() {
   const uint32_t t = HAL_GetTick();
   if (t - last_press_time < key_press_delay) return -1;
   int16_t ind = get_index();
   if (ind != last_index) {
      last_index = ind;
      if (ind != -1) {
         last_press_time = t;\
      }
      return ind;
   }
   return -1;
}
```

Main функция

```
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
HAL_TIM_Base_Start_IT(&htim6);
TIM6->ARR = duration*10;


while (1)
{
    //HAL_UART_Receive_IT(&huart6, (uint8_t *) s, sizeof( char ));
    //HAL_UART_Transmit_IT( &huart6, (uint8_t *) s, sizeof( s ));
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
 check_button();
 int ind = get_index_checked();
  if (ind != -1) {
     if (is_test) {
        char answer[100];
        sprintf(answer, "%d", ind+1);
        write(answer);
     }
     last_index = ind;
     char cur = keys[ind];
     process_symbol(cur);
  }
 if(is_writing_now == 0){
  if(transmit_from_pointer != write_to_pointer) {
     is_writing_now = 1;
     HAL_UART_Transmit_IT( &huart6, (uint8_t *) transmit_from_pointer, sizeof(
char ));
  }
 }
}
```

```
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance == huart6.Instance) {
       is_writing_now = 0;
        next(&transmit_from_pointer, write_buffer);
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
   if(htim->Instance == TIM6) {
      if (TIM1->CCR1 > 0) {
         note_index++;
         if (note_index < 7 && is_all_playing) {
            play(note_index);
            char answer[100];
            sprintf(answer, "Note: %s, octave: %d", note_name[note_index],
octave+1);
            write(answer);
         } else {
            note_index = 0;
            is_all_playing = 0;
            mute();
         }
      }
   }

}
```

Вывод

Во время выполнения лабораторной работы мы изучили работу интерфейса I2C и особенности передачи данных через него. Также мы получили навыки программирования для контроллера интерфейса и разработали программу, использующую его для считывания нажатий клавиатуры.